SYSTEMVERILOG · HOMEWORK

# FIFO TB

push_data 무작위 · 0~255 full coverage

tb_fifo_pre · fifo (DEPTH 4, 8 bit)

environment (heap)




posedge + 1 ns 구동 → negedge 판정 → gen_next_evt

push_cover / pop_cover == 256 → ->end_evt

**정안성**

2026. 11

구조

## gen → drv → DUT → mon → scb

mailbox 2개 · event 2개, virtual interface로 DUT 구동 · 관측

구현 1

## 기대 queue 판정

scoreboard 내부 queue (DEPTH 4)로 full · empty · pop_data를 매 cycle 비교

구현 2

## coverage 종료

0~255를 모두 확인하면 end_evt → report, run_count 제거 (최대 20,000)

결과

## 5 / 5 Option PASS

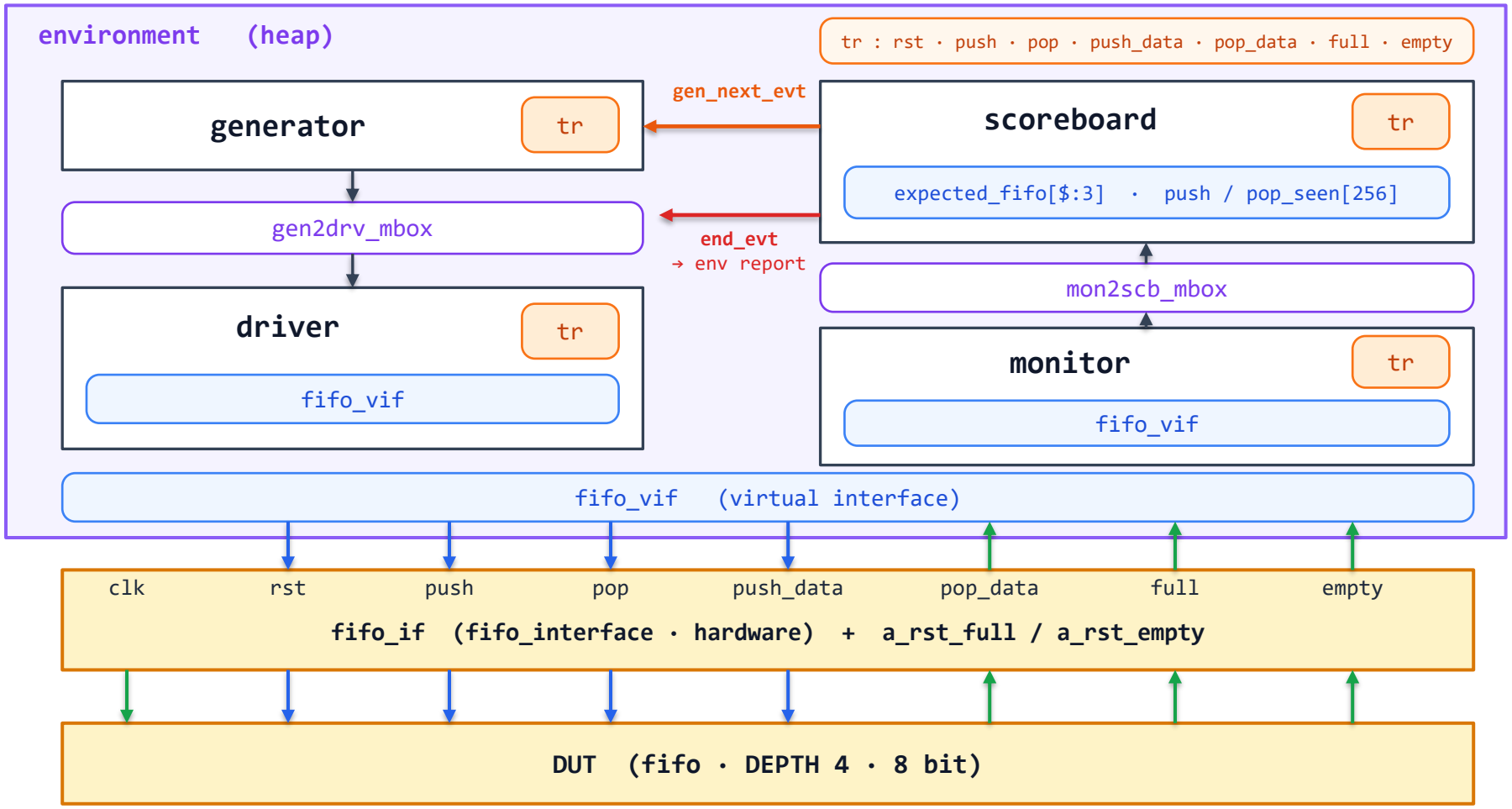
FAIL 0 · assertion 위반 0 · 트러블 슈팅 1건 해결 (data full coverage)

# 검증 환경: gen → drv → DUT → mon → scb 구조

01

**결론**

- generator가 만든 stimulus를 driver가 DUT에 넣고, monitor가 읽은 출력을 scoreboard가 기대 queue와 비교
- scoreboard가 판정 후 gen_next_evt로 다음 transaction 요청, 0~255를 모두 확인하면 end_evt로 종료




**generator**
· gen_tr(option)으로 stimulus 생성 → put
· @(gen_next_evt) 대기 → 다음 tr

**driver**
· preset : rst 1 → negedge 2번 → rst 0
· posedge + 1 ns 에 get → 4개 입력 구동

**monitor**
· negedge 에서 7개 신호 샘플
· mon2scb_mbox.put

**scoreboard**
· 기대 queue로 full · empty · pop_data 판정
· 0~255 확인 시 ->end_evt

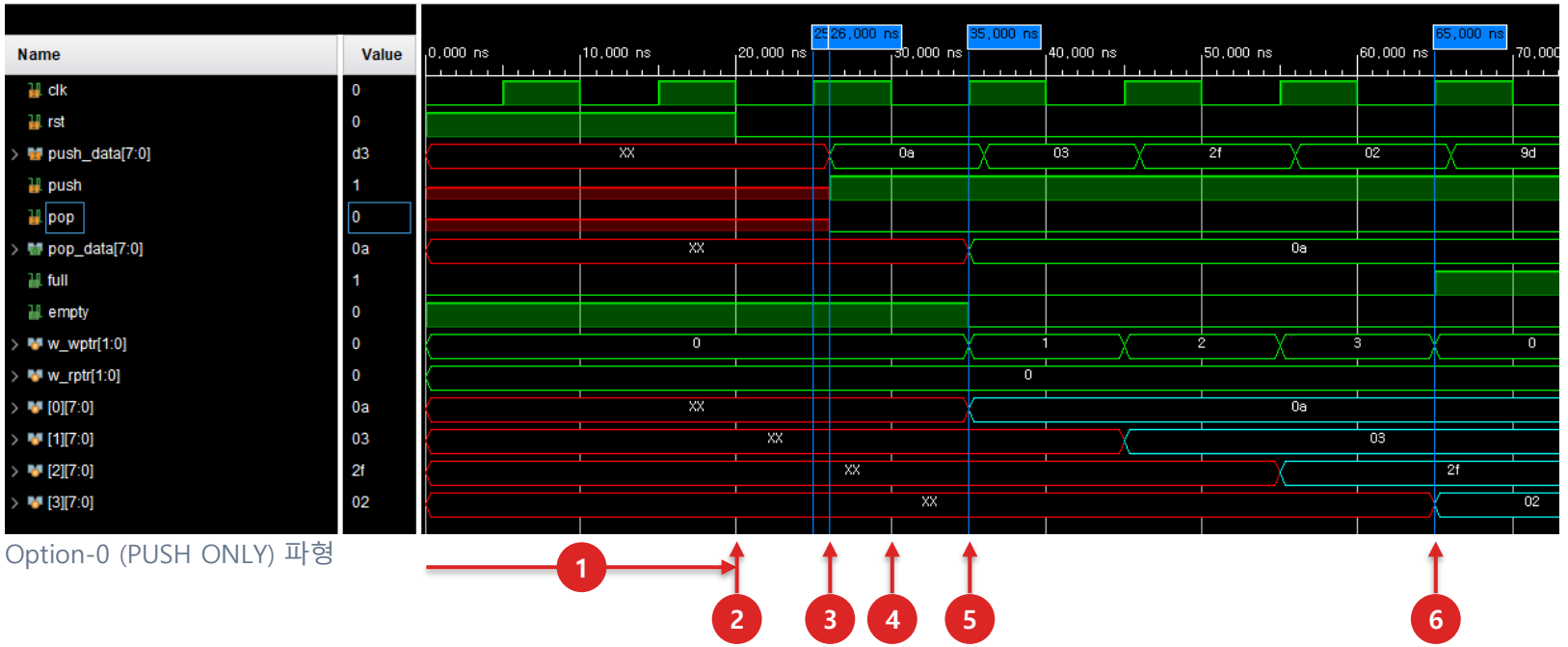
**environment**
· preset → fork … join_none
· @(end_evt) → #1 → report

# 검증 타이밍: preset 후 posedge + 1 ns 구동 → negedge 판정

01

**결론**

- **preset(0~20 ns) 후 매 cycle : driver 구동(posedge + 1 ns) → monitor 샘플 · scoreboard 판정(negedge)**
- **구동과 판정을 반 클럭 떨어뜨려 DUT 출력이 안정된 시점에 판정 → race 없음**




Option-0 (PUSH ONLY) 파형

1 **0~20 ns** preset : negedge 10 · 20 후 rst 0

2 **20 ns** gen : gen_tr → put

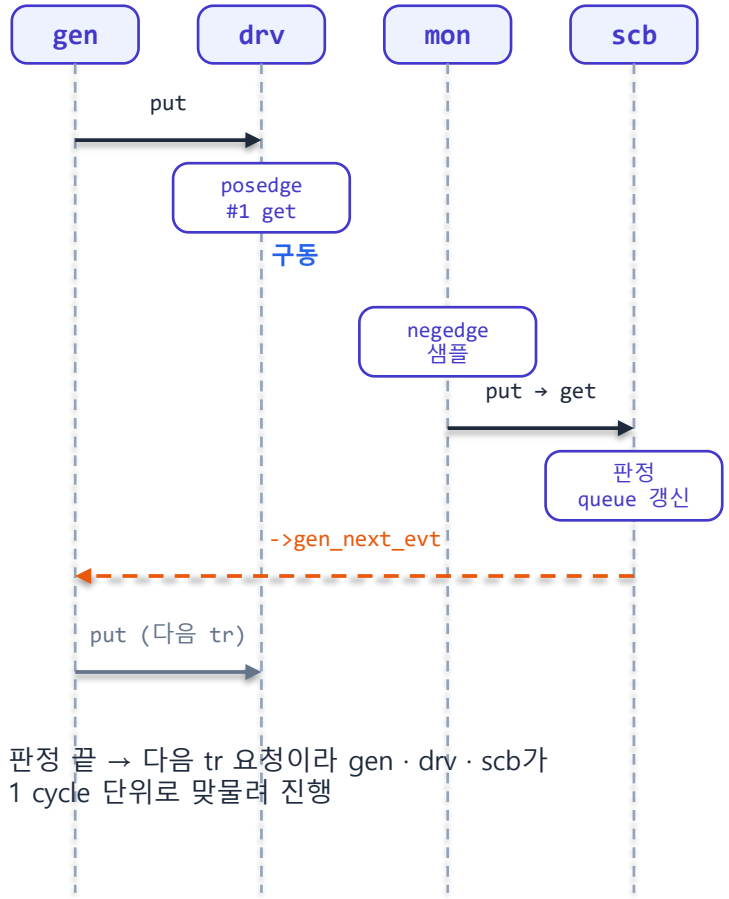
3 **26 ns** drv : posedge 25 + 1 ns 구동

4 **30 ns** mon · scb : negedge 샘플 · 판정

5 **35 ns** DUT : posedge 0a 저장, empty 0

6 **65 ns** DUT : 4개 저장 → full 1

### 1 transaction 흐름




판정 끝 → 다음 tr 요청이라 gen · drv · scb가
1 cycle 단위로 맞물려 진행

# 검증 Plan: 고정 패턴 3개 + 무작위 2개, 0~255 full coverage

02

결론

- Option-0~2는 정해진 패턴으로 경계 동작(full · empty · 동시 요청)을 하나씩, Option-3~4는 무작위로 조합 탐색
- 모든 Option은 push_data [0:255] 무작위, 0~255를 모두 확인할 때까지 실행 (coverage 종료, 최대 20,000)

| Option | stimulus | 기준 근거 | 예상 출력 | 종료 횟수 |
|---|---|---|---|---|
| **Option-0<br>PUSH ONLY** | 라운드 6 cycle<br>push 4 → push 1 (full) → rst | DEPTH 4라 reset 없이 push만 하면 4개 이후 전부 무시<br>→ 라운드마다 reset으로 비워 0~255 모두 저장 | 4개 저장 후 full = 1<br>5번째 push 무시, reset 후 empty = 1 | **2,101** |
| **Option-1<br>POP ONLY** | 라운드 9 cycle<br>push 4 → pop 5 | reset 직후 empty라 pop할 데이터가 없음<br>→ 4개 채운 뒤 pop 4 + underflow 1 | 넣은 순서대로 출력, 4번째 pop 후 empty = 1<br>5번째 pop 무시 | **2,906** |
| **Option-2<br>PUSH & POP** | 매 cycle push = 1, pop = 1 | 같은 cycle 동시 요청 (포인터 동시 이동)<br>empty에서는 push만 처리되는 예외 확인 | 첫 cycle 1개 저장 후 1개 유지<br>pop_data = 직전 push 값 | **1,746** |
| **Option-3<br>RANDOM** | push · pop 각 50%<br>rst = 0 고정 | 정해진 패턴에 없는 조합 (full에서 push + pop 등)<br>reset 영향 없이 데이터 순서만 확인 | pop_data = queue 맨 앞 값<br>full · empty = queue 개수 4 · 0 | **3,130** |
| **Option-4<br>RANDOM + rst** | rst 10%<br>push · pop 각 50% | 동작 중 비동기 reset 후 비워지고 재개되는지<br>10% = full 경계와 reset 빈도의 균형 | rst cycle : full 0, empty 1<br>저장 데이터 버림 (queue도 비움) | **4,622** |
| **공통** | preset : rst 1 → negedge 2번<br>push_data [0:255] 무작위 | 같은 초기 상태에서 시작해야 비교 가능<br>순차 입력이면 값 v → 항상 주소 v % 4 (고정 조합) | reset 직후 full 0, empty 1<br>(scoreboard + assertion) | - |

# Generator: Option별 stimulus를 step(라운드 순번)으로 생성

02

**결론**

- **Option-0 · 1은 step % 6, step % 9 로 라운드를 나눠 같은 패턴 반복, Option-2~4는 매 cycle 제약조건대로**
- **모든 Option은 먼저 randomize() → push_data [0:255] 무작위, 그 뒤 Option에 맞게 rst · push · pop 덮어씀**

class generator · gen_tr()

```
function void gen_tr(int option);
  case(option)
    0:begin
      tr.randomize();
      tr.rst  = 0;
      tr.pop  = 0;
      case(step % 6)
        0,1,2,3 : begin tr.push = 1; end
        4       : begin tr.push = 1; end
        5       : begin tr.push = 0; tr.rst = 1; end
      endcase
      step++;
    end
    1:begin
      tr.randomize();
      tr.rst  = 0;
      if(step % 9 < 4) begin tr.push = 1; tr.pop = 0; end
      else             begin tr.push = 0; tr.pop = 1; end
      step++;
    end
    2:begin
      tr.randomize();
      tr.rst = 0;  tr.push = 1;  tr.pop = 1;
    end
    3:begin
      tr.randomize();
      tr.rst = 0;
    end
    4:begin
      tr.randomize();
    end
  endcase
endfunction
```

1. **Option-0 : step % 6**
   0~3 push (4개 저장) → 4 push (full 무시)
   → 5 rst (비움)

2. **Option-1 : step % 9**
   0~3 push 4개 → 4~8 pop 5번
   (출력 4 + underflow 1)

3. **Option-2 : push = pop = 1**
   매 cycle 동시 요청, rst 0

4. **Option-3 : rst = 0**
   push · pop 50% 무작위, reset 없이

5. **Option-4 : 제약조건 그대로**
   rst 10% · push / pop 각 50%

**step** : 라운드 안의 cycle 순번, transaction 1개마다 1 증가

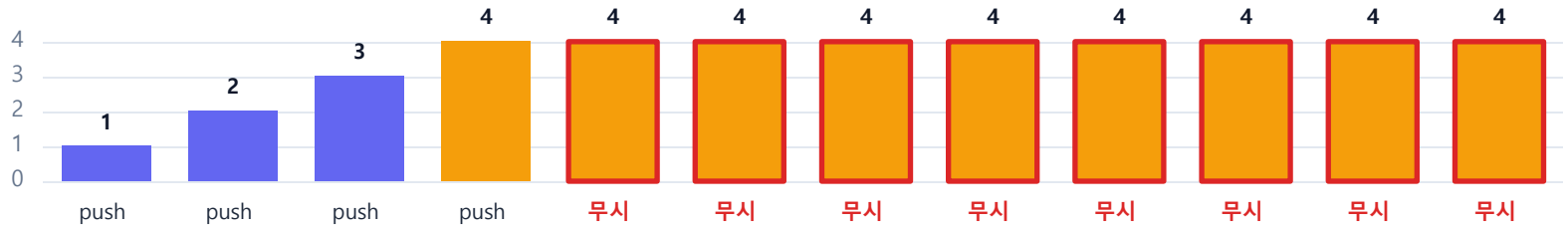
# Option-0 PUSH ONLY: 저장 4 → full 무시 → reset 반복

02

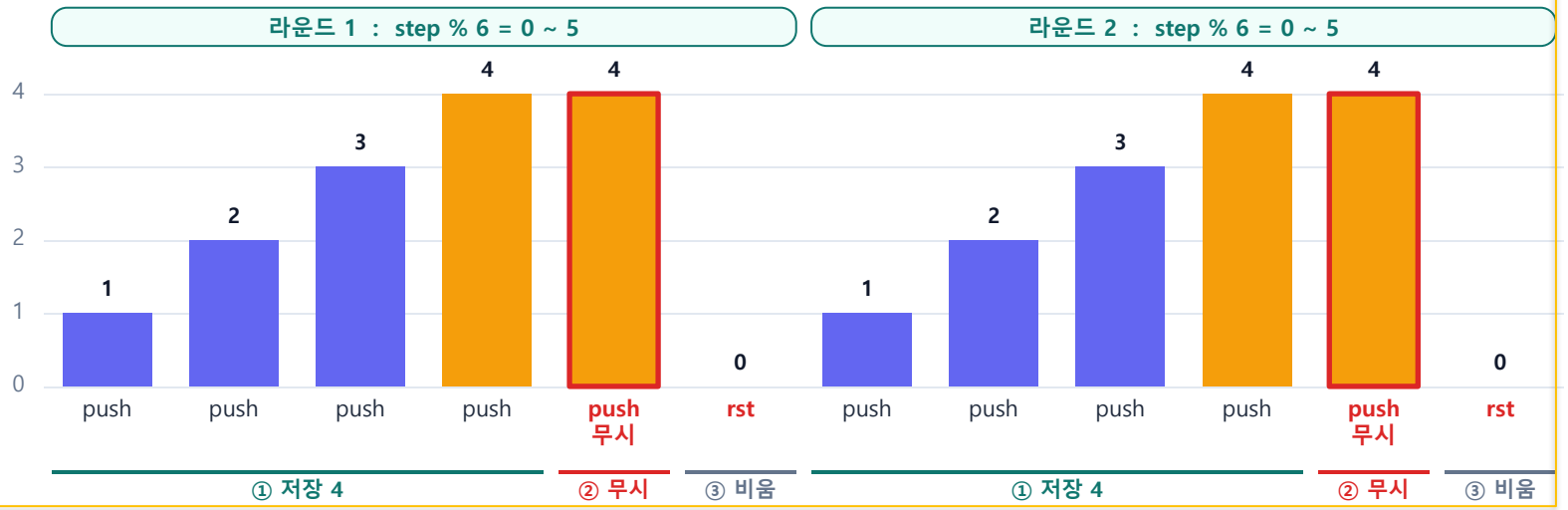
**결론**

- **DEPTH 4라 reset 없이 push만 하면 4개 이후 모든 push가 무시돼 0~255 중 4개 값밖에 저장 못 함**
- **라운드(6 cycle)마다 reset으로 비워 full 진입 · full 무시 · reset 비움을 반복 확인하며 0~255 모두 저장**

## reset 없이 push만 할 때




## PUSH ONLY 시나리오




**① 저장 4 cycle = DEPTH**
reset 후 empty에서 4번 push → full 진입
0~255 무작위 값이 매 라운드 4개씩 저장

**② push 1 cycle = full 무시**
full 상태 push가 무시되는지 (overflow 방지)
한 번이면 확인 충분 → 1 cycle

**③ rst 1 cycle = 비움**
queue · DUT 모두 비움 → 다음 라운드에
새 값 4개를 다시 저장할 수 있음

**결과 (2,101 cycle)**
라운드 350회
push 저장 1,401 · 무시 350
**저장된 push_data 256 / 256**

# Option-1 POP ONLY: 채움 4 → pop 4 → underflow 1

02

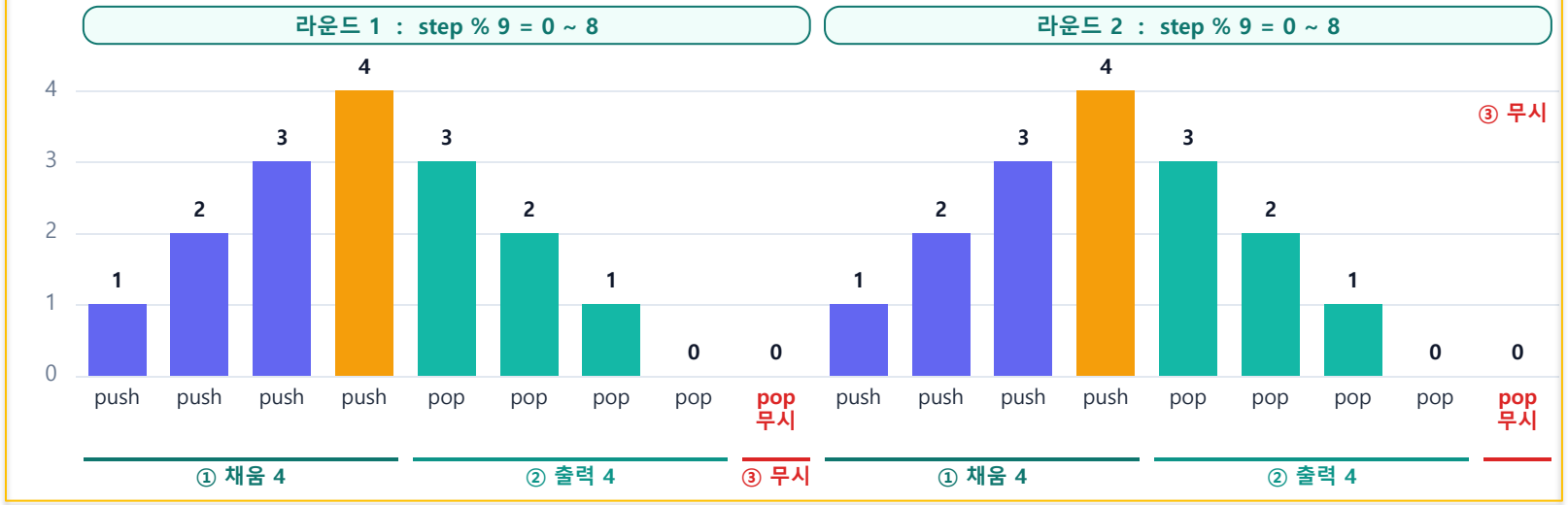
결론

- reset 직후는 empty라 pop만 하면 꺼낼 데이터가 없음 → 라운드마다 먼저 4개를 채운 뒤 pop만 5번
- pop으로 비우므로 reset 없이 다음 라운드 시작, empty 진입과 empty 상태 pop 무시를 매 라운드 확인

## 채우지 않고 pop만 할 때 (reset 직후 empty)




## PUSH ONLY 시나리오




**1** ① push 4 cycle = DEPTH
pop할 데이터를 먼저 채움
4번째 push에서 full 진입

**2** ② pop 4 cycle = 순서 확인
넣은 4개가 같은 순서로 나오는지
(queue pop_front 값과 비교)

**3** ③ pop 1 cycle = underflow
empty 상태 pop이 무시되는지
→ 비어 있으므로 reset 불필요

**결과 (2,906 cycle)**
push 1,292 · pop 1,292
empty 상태 pop 무시 322
pop 확인 (PASS) 256 / 256

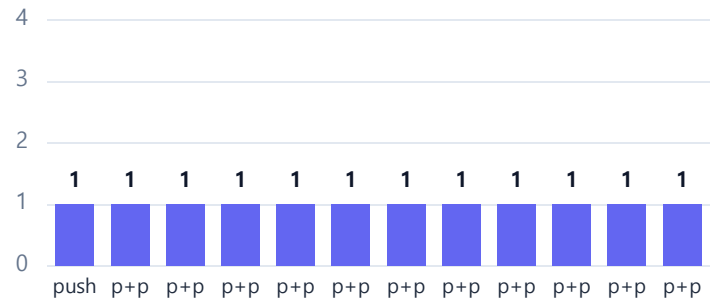
# Option-2 ~ 4: 동시 요청 · 무작위 조합 · 동작 중 reset

02

결론

- Option-0 · 1은 경계 동작을 하나씩 확인하는 고정 라운드, Option-2~4는 라운드 없이 매 cycle 같은 규칙으로 생성
- stage 기준 : 확인할 경계(full · empty · 무시)가 정해진 패턴은 라운드로 나누고, 조합 탐색은 무작위에 맡김

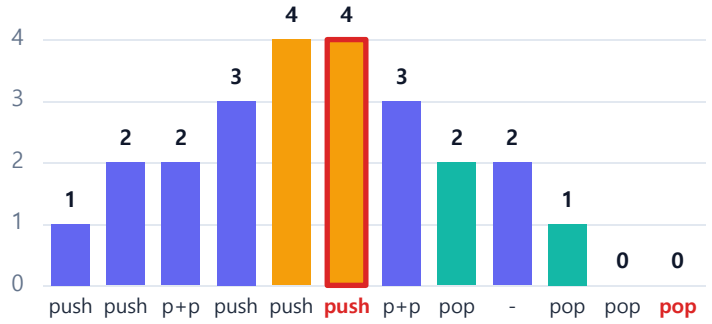
## Option-2 PUSH & POP




- 첫 cycle : empty라 push만 처리 (예외)
- 이후 push + pop → 개수 1 유지
- 매 cycle 직전 push 값이 pop_data로 나옴
- → 쓰기 → 읽기 데이터 경로를 매 cycle 확인

**확인 : pop_data = 직전 push 값**

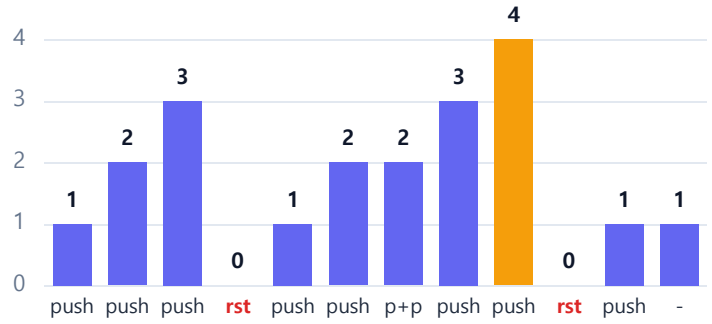
## Option-3 RANDOM (예시)




- full에서 push → 무시, full에서 p+p → pop만
- empty에서 pop → 무시
- 개수 0~4 사이 임의 전이를 무작위로 탐색
- rst = 0 : reset 없이 데이터 순서만 확인

**확인 : full · empty 경계, 동시 요청 예외**

## Option-4 RANDOM + rst (예시)




- 데이터가 남은 상태에서 rst → 즉시 0
- 남은 데이터가 pop되지 않는지 확인
- rst 10% : 너무 잦으면 full까지 못 참
- 너무 드물면 reset 상황이 부족 → 균형

**확인 : rst 후 full 0 · empty 1, 데이터 버림**

# #1 check_flags: queue 기대값과 DUT full · empty를 매 cycle 비교

03

**결론**

- **scoreboard 내장 함수 : 기대 full · empty를 인자로 받아 DUT의 tr.full · tr.empty와 비교 → pass / fail 누적**
- **=== (case 등가)로 비교 → DUT 출력이 x · z여도 비교 결과가 0 / 1로 확정되어 FAIL로 집계**

**class scoreboard · check_flags()**

```
function void check_flags(bit expec_full, bit expec_empty);
  if(tr.full === expec_full) full_pass_count ++;                    1
  else begin
    full_fail_count ++;
    $display("%t  FULL FAIL  : full = %b / expected_full = %b (queue size = %0d)",
             $time, tr.full, expec_full, expected_fifo.size());     3
  end

  if(tr.empty === expec_empty) empty_pass_count ++;                 2
  else begin
    empty_fail_count ++;
    $display("%t  EMPTY FAIL : empty = %b / expected_empty = %b (queue size = %0d)",
             $time, tr.empty, expec_empty, expected_fifo.size());
  end
endfunction
```

**1** **tr.full === expec_full**
같으면 full_pass_count, 다르면 full_fail_count

**2** **tr.empty === expec_empty**
empty도 같은 방식으로 따로 집계

**3** **FAIL 시 시간 · queue size 출력**
어느 cycle, 어떤 개수에서 틀렸는지 바로 확인

### 호출 위치 · 기대값

| 분기 | 호출 | 기대값 |
|---|---|---|
| tr.rst = 1 | check_flags(0, 1) | full 0 · empty 1 (비워짐) |
| tr.rst = 0 | check_flags(size == 4, size == 0) | queue 개수로 계산 |

**== 대신 === 를 쓴 이유**

== : 한쪽이 x · z면 비교 결과도 x
=== : x · z까지 그대로 비교 → 결과가 항상 0 / 1
**→ DUT full · empty가 x면 확실히 FAIL로 집계**
assertion도 같은 이유로 === 사용

# #2-1 run task (rst): DUT reset을 받으면 기대 queue도 같이 비움

03

**결론**

- monitor가 넘긴 tr.rst = 1이면 queue를 비우고 reset 직후 기대값 full 0 · empty 1로 비교
- reset 직후 상태는 scoreboard 비교 + interface assertion 두 방식으로 확인

### class scoreboard · run()

```
task run();
  forever begin
    mon2scb_mbox.get(tr);
    total_count ++;

    if(tr.rst)begin
      reset_count ++;          1
      expected_fifo.delete();  2
      check_flags(0, 1);       3
      cg_fifo.sample();        4
    end
    else begin
      ...
    end
```

1 **reset_count ++**
reset 횟수 집계 (Option-0 · 4에서 사용)

2 **expected_fifo.delete()**
DUT와 같이 기대 queue를 비움
→ reset 전 데이터는 더 이상 기대하지 않음

3 **check_flags(0, 1)**
reset 직후 기대값 full 0 · empty 1 비교

4 **cg_fifo.sample()**
reset cycle도 coverage에 집계

### interface fifo_interface · assertion

```
property p_rst_full;
  @(posedge clk) rst |-> (full === 1'b0);
endproperty
property p_rst_empty;
  @(posedge clk) rst |-> (empty === 1'b1);
endproperty

a_rst_full  : assert property(p_rst_full)
  else $display("%t : [assert] error full = %d", $time, full);
a_rst_empty : assert property(p_rst_empty)
  else $display("%t : [assert] error empty = %d", $time, empty);
```

5

5 **a_rst_full / a_rst_empty**
rst 동안 매 posedge full === 0, empty === 1
scoreboard와 별개로 interface에서 확인
결과 : 모든 Option assertion 위반 0

# #2-2 run task: queue를 갱신하기 전에 DUT full · empty와 비교

03

**결론**

- **scoreboard는 negedge에 판정 → queue는 판정 직후 갱신되어 DUT 메모리보다 반 클럭 빠름**
- **그래서 queue 갱신 전 개수로 기대값을 계산해 비교한 뒤 pop / push를 반영**

**class scoreboard · run() else**

```
else begin
  exp_full  = (expected_fifo.size() == DEPTH);
  exp_empty = (expected_fifo.size() == 0);
  check_flags(exp_full, exp_empty);
  cg_fifo.sample();

  do_pop  = tr.pop  && !exp_empty;
  do_push = tr.push && !exp_full;
  ...  // pop_front / push_back (queue 갱신)
end
```

1

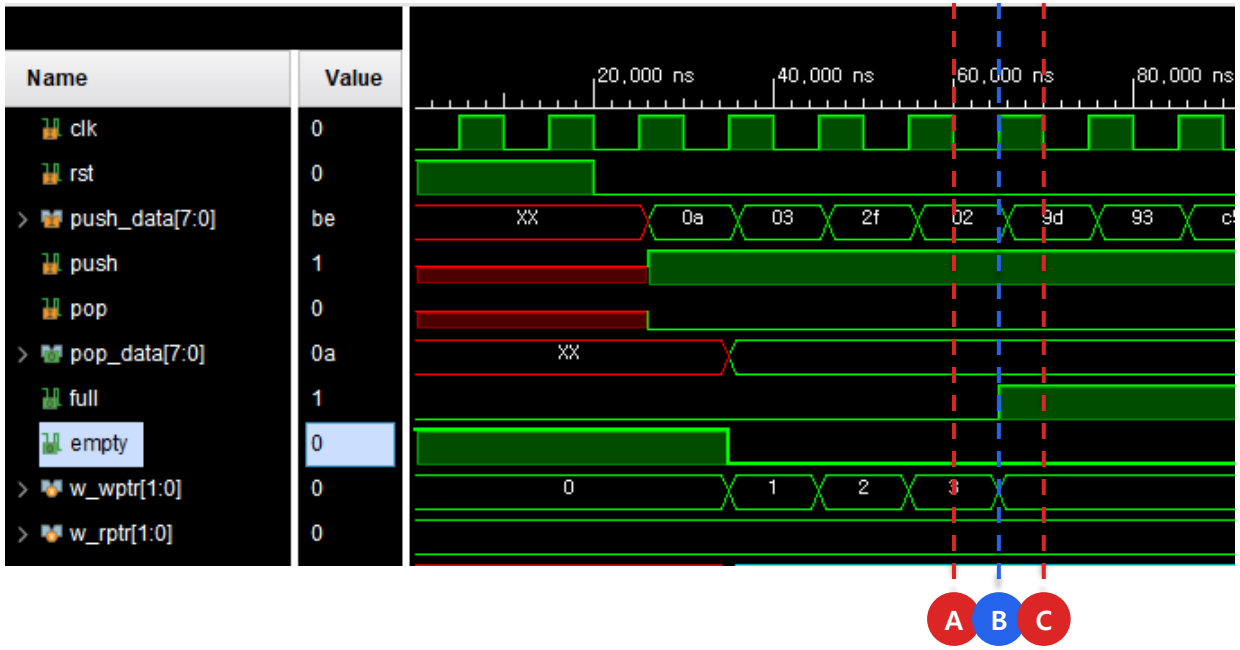
2

**A** **60 ns negedge · scb 판정**
DUT 3개 → full 0 / queue 3개 → exp_full 0 ✓
→ 비교 후 queue push (queue 4개)

**B** **65 ns posedge · DUT**
4번째 값 저장 → full 1
(queue는 이미 A에서 4개 = 반 클럭 빠름)

**C** **70 ns negedge · scb 판정**
DUT full 1 / queue 4개 → exp_full 1 ✓
→ 5번째 push는 무시

**Option-0 파형 · 4번째 push (02)**




**만약 queue를 먼저 갱신하면**
A에서 queue 4개 → exp_full 1, DUT full 0
**→ 정상 동작인데 FULL FAIL**

# #2-3 run task: queue 상태 × 요청 → do_pop / do_push 결정

03

**결론**

- DUT의 full · empty가 아니라 queue의 exp_full · exp_empty로 판단 → DUT가 틀려도 기대값은 흔들리지 않음
- 요청했지만 처리되지 않은 push / pop은 ignore count로 따로 집계 (report에 출력)

**class scoreboard · run()**

```
do_pop  = tr.pop  && !exp_empty;
do_push = tr.push && !exp_full;

if(tr.push && exp_full)  push_ignored_count ++;
if(tr.pop  && exp_empty) pop_ignored_count ++;
```

1

2

**1 queue 상태 × 요청 → 처리 여부**

DUT의 full · empty를 쓰면 DUT에 오류가 있을 때
기대값도 같이 틀려져 정확한 판별이 안 됨
→ queue(기대 모델) 기준으로 판단

**2 push / pop ignored count**

full 상태 push, empty 상태 pop 횟수
→ 무시 동작이 실제로 몇 번 확인됐는지 report로 확인

**처리 결과 표 (queue 개수 기준)**

| 요청 | empty (0개) | 1 ~ 3개 | full (4개) |
|---|---|---|---|
| push | push | push | 무시 |
| pop | 무시 | pop | pop |
| push + pop | push만 | push + pop | pop만 |
| idle | - | - | - |

- 빨간 칸 = 요청과 실제 처리가 다른 경계 상황
- **push만 / pop만** : 동시 요청에서 한쪽만 처리되는 예외
- DUT도 같은 규칙이어야 queue와 결과가 일치

# #2-4 run task: pop → 맨 앞과 비교, push → 맨 뒤에 저장

03

**결론**

- pop_front()로 꺼낸 기대값과 DUT pop_data를 === 비교 → PASS한 값만 pop_seen[256]에 기록
- push_back()으로 저장한 값은 push_seen[256]에 기록, 처음 보는 값일 때만 cover 수 증가 (중복 집계 X)

**class scoreboard · run()**

```
if(do_pop)begin
  pop_count ++;
  exp_data = expected_fifo.pop_front();                                    1
  if(exp_data === tr.pop_data)begin
    pass_count ++;
    if(!pop_seen[exp_data]) begin pop_seen[exp_data] = 1; pop_cover ++; end   2
  end
  else begin
    fail_count ++;
    $display("%t  POP FAIL : pop_data = %d / expected_pop_data = %d",      3
             $time, tr.pop_data, exp_data);
  end
end

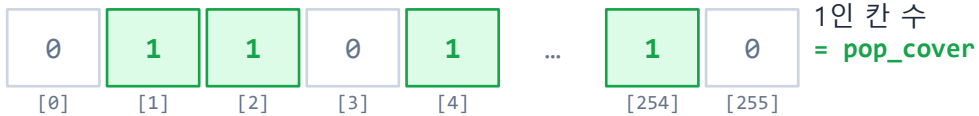
if(do_push)begin
  push_count ++;
  expected_fifo.push_back(tr.push_data);
  if(!push_seen[tr.push_data]) begin push_seen[tr.push_data] = 1; push_cover ++; end   4
end
```

1. **pop_front() → === 비교**
   queue 맨 앞 = 가장 먼저 넣은 값 → FIFO 순서 확인
2. **PASS한 값만 pop_seen**
   처음 보는 값이면 pop_cover ++ (0~255 중 확인된 값 수)
3. **FAIL → 실제 · 기대값 출력**
   fail_count ++, 어느 시간에 어떤 값이 틀렸는지 표시
4. **push_back() + push_seen**
   queue 맨 뒤에 저장, 처음 저장된 값이면 push_cover ++

**expected_fifo (최대 4개)**




**pop_seen[256] · pop_cover**



# #2-5 run task: 0~255 확인 시 end_evt, 안전장치 max_count

03

**결론**

- Option-0은 pop이 없으므로 push_cover, 나머지는 pop_cover(저장 + 출력까지 확인)가 256이 되면 ->end_evt
- max_count(20,000)는 DUT 오류로 영영 256이 안 될 때 무한 실행을 막는 안전장치, 판정 후 ->gen_next_evt

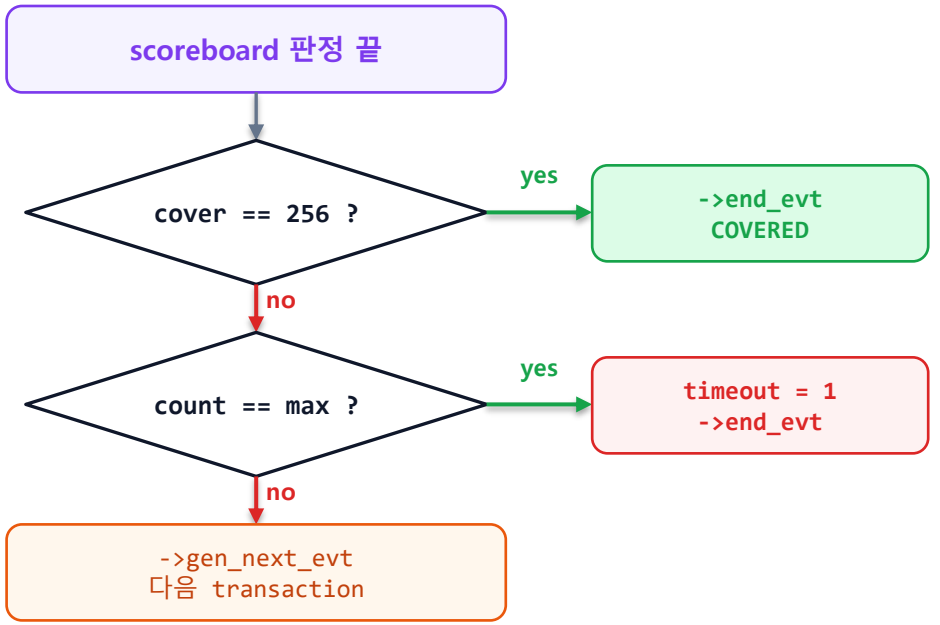
**class scoreboard · run() 끝**

```
if(((option == 0) ? push_cover : pop_cover) == 256) ->end_evt;   1
else if(total_count == max_count) begin timeout = 1; ->end_evt; end   2

->gen_next_evt;   3
```

**class environment · run()**

```
task run(int max_count,int option);
  drv.preset();
  scb.option    = option;
  scb.max_count = max_count;
  fork
    gen.run(option);
    drv.run();
    mon.run();
    scb.run();
  join_none                    4
  @(scb.end_evt);
  #1;                          5
  scb.report(option);
  disable fork;
endtask
```




1. **Option-0 → push_cover**
   pop이 없어 저장 기준
2. **max → TIMEOUT**
   report에 TIMEOUT 표시
3. **->gen_next_evt**
   판정 후 다음 tr 요청
4. **join_none**
   gen이 forever라 end_evt로만 종료
5. **#1 → report**
   마지막 판정 반영 후 집계, disable fork

# #3 covergroup · report: 값 · 요청 · 상태 조합을 coverage로 집계

03

결론

- push_data · pop_data 0~255, 요청 4종, flag 3종, 데이터 개수 5종과 그 조합(cross)을 매 cycle sample
- sample()은 check_flags 직후, queue 갱신 전 → 요청이 들어온 시점의 상태로 집계

| coverpoint | 대상 | bins | 의미 |
|---|---|---|---|
| cp_push_data | push_data iff(push) | 256 | push된 값 0~255 |
| cp_pop_data | pop_data iff(pop) | 256 | pop된 값 0~255 |
| cp_push_pop | {push, pop} | 4 | idle · push · pop · push+pop |
| cp_empty_full | {empty, full} | 3 | normal · empty · full |
| cp_queue_st | expected_fifo.size() | 5 | 데이터 개수 0 ~ 4 |
| cx_op_st | push_pop × empty_full | 12 | 요청 × flag 상태 |
| cx_op_st_fifo_depth | push_pop × queue_st | 20 | 요청 × 데이터 개수 |

```
+============================================================+
|  [Option-3] RANDOM without Reset TEST          PASS      |
+============================================================+
|  TRANSACTION                                               |
+------------------------------------------------------------+
|    total                        :  3130                    |
|    reset                        :     0                    |
|    push (accepted)              :  1385                    |
|    pop  (accepted)              :  1383                    |
|    push ignored (full)          :   176                    |
|    pop  ignored (empty)         :   179                    |
+------------------------------------------------------------+
|  CHECK                                                     |
+------------------------------------------------------------+
|    pop_data vs expected         : PASS  1383   FAIL   0    |
|    full                         : PASS  3130   FAIL   0    |
|    empty                        : PASS  3130   FAIL   0    |
+------------------------------------------------------------+
|  DATA ACCEPT (0~255)                                       |
+------------------------------------------------------------+
|    end condition                : COVERED                  |
|    transactions                 :  3130 / max 20000        |
|    push accepted (stored)       : 256 / 256                |
|    pop  checked  (PASS)         : 256 / 256                |
+------------------------------------------------------------+
|  COVERAGE                                                  |
+------------------------------------------------------------+
|    push_data                    : 100.0 %                  |
|    pop_data                     : 100.0 %                  |
|    push / pop                   : 100.0 %                  |
|    empty / full                 : 100.0 %                  |
|    fifo count (0~3)             : 100.0 %                  |
|    push/pop x empty/full        : 100.0 %                  |
|    Depth x Push / Pop           : 100.0 %                  |
+------------------------------------------------------------+
```

- report는 get_coverage()로 coverpoint별 % 출력
- xsim 2020.2는 %-28s 왼쪽 정렬을 무시 → pad()로 공백을 채워 칸을 맞춤

# 트러블 슈팅 #1: run_count 고정으로 data full coverage 보장 X

04

**결론**

- run_count 1000으로 실행하면 Option마다 0~255 중 일부 값이 한 번도 확인되지 않음 (Option-4 : pop 79개 누락)
- 원인 : constraint는 범위만 정할 뿐 모든 값이 나온다는 보장이 없고, 횟수를 검증자가 임의로 고정

### 수정 전 · Option-4 (run_count 1000)

```
+============================================================+
|  [Option-4] RANDOM with Reset TEST              PASS       |
+============================================================+
|  TRANSACTION                                               |
+------------------------------------------------------------+
|    total                        :  1000                    |
|    reset                        :   114                    |
|    push (accepted)              :   414                    |
|    pop  (accepted)              :   292                    |
|    push ignored (full)          :    12                    |
|    pop  ignored (empty)         :   153                    |
+------------------------------------------------------------+
|  CHECK                                                     |
+------------------------------------------------------------+
|    pop_data vs expected         : PASS   292   FAIL   0    |
|    full                         : PASS  1000   FAIL   0    |
|    empty                        : PASS  1000   FAIL   0    |
+------------------------------------------------------------+
|  MISSING DATA CHECK (0~255)                                |
+------------------------------------------------------------+
|    push accepted (stored)       : 215 / 256                |
|    pop  checked  (PASS)         : 177 / 256                |
+------------------------------------------------------------+
|  COVERAGE                                                  |
+------------------------------------------------------------+
|    push_data                    :  84.8 %                  |
|    pop_data                     :  74.6 %                  |
|    push / pop                   : 100.0 %                  |
|    empty / full                 : 100.0 %                  |
|    fifo count (0~3)             : 100.0 %                  |
|    push/pop x empty/full        : 100.0 %                  |
+============================================================+
```

1

1

### 원인

**class transaction**

```
constraint push_data_gen{push_data inside{[0:255]};}
```

2

**generator (수정 전)**

```
task run(int run_count,int option);
  repeat(run_count)begin
    ...
  end
endtask
```

3

**tb top (수정 전)**

```
initial begin
  env = new(fifo_if);
  env.run(1000,4);
  $finish;
end
```

3

2 **inside [0:255]**
값의 범위만 정함
매번 독립적으로 뽑으므로
같은 값이 반복되고
안 나오는 값도 생김

3 **run_count = 1000 고정**
얼마나 돌려야 다 나오는지
기준이 없음
0~255를 모두 뽑는 평균 횟수
256 × (1 + 1/2 + … + 1/256) ≈ 1,568

# 해결: run_count 제거 → 0~255 확인 시 end_evt로 자동 종료

04

결론

- scoreboard가 seen[256] 배열로 확인된 값 수(cover)를 세고, 256이 되면 ->end_evt
- generator는 forever, environment는 fork join_any → join_none + @(scb.end_evt)로 종료 시점 결정

## 수정 전 · tb_fifo.sv

### generator

```
task run(int run_count,int option);
  repeat(run_count)begin
```

### environment

```
task run(int run_count,int option);
   drv.preset();
   fork
     gen.run(run_count,option);
     drv.run();
     mon.run();
     scb.run();
   join_any
   #1;
   scb.report(option);
   disable fork;
endtask
```

## 수정 후 · tb_fifo_pre.sv

### generator

```
task run(int option);
  forever begin
```

1

### scoreboard

```
if(((option == 0) ? push_cover : pop_cover) == 256) ->end_evt;
else if(total_count == max_count) begin timeout = 1; ->end_evt; end
```

2

### environment

```
task run(int max_count,int option);
   drv.preset();
   scb.option    = option;
   scb.max_count = max_count;
   fork
     gen.run(option);
     drv.run();
     mon.run();
     scb.run();
   join_none
   @(scb.end_evt);
   #1;
   scb.report(option);
   disable fork;
endtask
```

3

4

1. **repeat → forever**
   횟수 제한 제거, 종료는 scoreboard가 결정
2. **seen[256] + end_evt**
   0~255 확인 수가 256이면 ->end_evt
   max_count에서는 TIMEOUT
3. **run_count → max_count**
   횟수 지정이 아니라 안전장치 상한
4. **join_any → join_none**
   forever라 join_any가 끝나지 않음
   → @(scb.end_evt)로 대기

# 적용 결과 Option-0 PUSH ONLY: 239 → 256 / 256

04

**결론**

- 수정 전 run_count 1,000 : 저장된 push_data 239 / 256 → 17개 누락
- 수정 후 2,101 cycle에서 256 / 256 확인 후 자동 종료 (COVERED), FAIL 0

## 수정 전 · tb_fifo.sv

```
+=========================================================+
|  [Option-0] PUSH ONLY TEST                      PASS    |
+=========================================================+
|  TRANSACTION                                            |
+---------------------------------------------------------+
|    total                        :  1000                 |
|    reset                        :   166                 |
|    push (accepted)              :   668                 |
|    pop  (accepted)              :     0                 |
|    push ignored (full)          :   166                 |
|    pop  ignored (empty)         :     0                 |
+---------------------------------------------------------+
|  CHECK                                                  |
+---------------------------------------------------------+
|    pop_data vs expected         : PASS     0   FAIL   0 |
|    full                         : PASS  1000   FAIL   0 |
|    empty                        : PASS  1000   FAIL   0 |
+---------------------------------------------------------+
|  MISSING DATA CHECK (0~255)                             |
+---------------------------------------------------------+
|    push accepted (stored)       : 239 / 256             |
|    pop  checked  (PASS)         :   0 / 256             |
+---------------------------------------------------------+
|  COVERAGE                                               |
+---------------------------------------------------------+
|    push_data                    :  96.1 %               |
|    pop_data                     :   0.0 %               |
|    push / pop                   :  25.0 %               |
|    empty / full                 : 100.0 %               |
|    fifo count (0~3)             :  75.0 %               |
|    push/pop x empty/full        :  25.0 %               |
+=========================================================+
```

## 수정 후 · tb_fifo_pre.sv

```
+=========================================================+
|  [Option-0] PUSH ONLY TEST                      PASS    |
+=========================================================+
|  TRANSACTION                                            |
+---------------------------------------------------------+
|    total                        :  2101                 |
|    reset                        :   350                 |
|    push (accepted)              :  1401                 |
|    pop  (accepted)              :     0                 |
|    push ignored (full)          :   350                 |
|    pop  ignored (empty)         :     0                 |
+---------------------------------------------------------+
|  CHECK                                                  |
+---------------------------------------------------------+
|    pop_data vs expected         : PASS     0   FAIL   0 |
|    full                         : PASS  2101   FAIL   0 |
|    empty                        : PASS  2101   FAIL   0 |
+---------------------------------------------------------+
|  DATA ACCEPT (0~255)                                    |
+---------------------------------------------------------+
|    end condition                : COVERED               |
|    transactions                 :  2101 / max 20000     |
|    push accepted (stored)       : 256 / 256             |
|    pop  checked  (PASS)         :   0 / 256             |
+---------------------------------------------------------+
|  COVERAGE                                               |
+---------------------------------------------------------+
|    push_data                    : 100.0 %               |
|    pop_data                     :   0.0 %               |
|    push / pop                   :  50.0 %               |
|    empty / full                 : 100.0 %               |
|    fifo count (0~3)             : 100.0 %               |
|    push/pop x empty/full        :  33.3 %               |
|    Depth x Push / Pop           :  30.0 %               |
+=========================================================+
```

**239 → 256**

저장된 push_data (0 ~ 255)

**17 → 0**

누락된 값

**1,000 → 2,101**

총 transaction (종료 시점)

pop이 없어 push 저장 기준으로 종료
reset 350회 (라운드 수)

# 적용 결과 Option-1 POP ONLY: 216 → 256 / 256

04

**결론**

- 수정 전 run_count 1,000 : pop 확인 (PASS) 216 / 256 → 40개 누락
- 수정 후 2,906 cycle에서 256 / 256 확인 후 자동 종료 (COVERED), FAIL 0

## 수정 전 · tb_fifo.sv

```
+=========================================================+
|  [Option-1] POP ONLY TEST                       PASS    |
+=========================================================+
|  TRANSACTION                                            |
+---------------------------------------------------------+
|    total                        :  1000                 |
|    reset                        :     0                 |
|    push (accepted)              :   445                 |
|    pop  (accepted)              :   444                 |
|    push ignored (full)          :     0                 |
|    pop  ignored (empty)         :   111                 |
+---------------------------------------------------------+
|  CHECK                                                  |
+---------------------------------------------------------+
|    pop_data vs expected         : PASS   444   FAIL   0 |
|    full                         : PASS  1000   FAIL   0 |
|    empty                        : PASS  1000   FAIL   0 |
+---------------------------------------------------------+
|  MISSING DATA CHECK (0~255)                             |
+---------------------------------------------------------+
|    push accepted (stored)       : 216 / 256             |
|    pop  checked  (PASS)         : 216 / 256             |
+---------------------------------------------------------+
|  COVERAGE                                               |
+---------------------------------------------------------+
|    push_data                    :  84.4 %               |
|    pop_data                     :  84.4 %               |
|    push / pop                   :  50.0 %               |
|    empty / full                 : 100.0 %               |
|    fifo count (0~3)             : 100.0 %               |
|    push/pop x empty/full        :  41.7 %               |
+=========================================================+
```

## 수정 후 · tb_fifo_pre.sv

```
+=========================================================+
|  [Option-1] POP ONLY TEST                       PASS    |
+=========================================================+
|  TRANSACTION                                            |
+---------------------------------------------------------+
|    total                        :  2906                 |
|    reset                        :     0                 |
|    push (accepted)              :  1292                 |
|    pop  (accepted)              :  1292                 |
|    push ignored (full)          :     0                 |
|    pop  ignored (empty)         :   322                 |
+---------------------------------------------------------+
|  CHECK                                                  |
+---------------------------------------------------------+
|    pop_data vs expected         : PASS  1292   FAIL   0 |
|    full                         : PASS  2906   FAIL   0 |
|    empty                        : PASS  2906   FAIL   0 |
+---------------------------------------------------------+
|  DATA ACCEPT (0~255)                                    |
+---------------------------------------------------------+
|    end condition                : COVERED               |
|    transactions                 :  2906 / max 20000     |
|    push accepted (stored)       : 256 / 256             |
|    pop  checked  (PASS)         : 256 / 256             |
+---------------------------------------------------------+
|  COVERAGE                                               |
+---------------------------------------------------------+
|    push_data                    : 100.0 %               |
|    pop_data                     : 100.0 %               |
|    push / pop                   :  50.0 %               |
|    empty / full                 : 100.0 %               |
|    fifo count (0~3)             : 100.0 %               |
|    push/pop x empty/full        :  41.7 %               |
|    Depth x Push / Pop           :  45.0 %               |
+=========================================================+
```

**216 → 256**

pop 확인 (PASS) (0 ~ 255)

**40 → 0**

누락된 값

**1,000 → 2,906**

총 transaction (종료 시점)

pop 확인 기준, empty 상태 pop 무시 322회도 함께 확인

# 적용 결과 Option-2 PUSH & POP: 253 → 256 / 256

04

**결론**

- 수정 전 run_count 1,000 : pop 확인 (PASS) 253 / 256 → 3개 누락
- 수정 후 1,746 cycle에서 256 / 256 확인 후 자동 종료 (COVERED), FAIL 0

## 수정 전 · tb_fifo.sv

```
+=========================================================+
|  [Option-2] PUSH & POP TEST                    PASS     |
+=========================================================+
|  TRANSACTION                                            |
+---------------------------------------------------------+
|    total                      :  1000                   |
|    reset                      :     0                   |
|    push (accepted)            :  1000                   |
|    pop  (accepted)            :   999                   |
|    push ignored (full)        :     0                   |
|    pop  ignored (empty)       :     1                   |
+---------------------------------------------------------+
|  CHECK                                                  |
+---------------------------------------------------------+
|    pop_data vs expected       : PASS   999   FAIL   0   |
|    full                       : PASS  1000   FAIL   0   |
|    empty                      : PASS  1000   FAIL   0   |
+---------------------------------------------------------+
|  MISSING DATA CHECK (0~255)                             |
+---------------------------------------------------------+
|    push accepted (stored)     : 253 / 256               |
|    pop  checked  (PASS)       : 253 / 256               |
+---------------------------------------------------------+
|  COVERAGE                                               |
+---------------------------------------------------------+
|    push_data                  :  98.8 %                 |
|    pop_data                   :  98.8 %                 |
|    push / pop                 :  25.0 %                 |
|    empty / full               :  66.7 %                 |
|    fifo count (0~3)           :  25.0 %                 |
|    push/pop x empty/full      :  16.7 %                 |
+=========================================================+
```

## 수정 후 · tb_fifo_pre.sv

```
+=========================================================+
|  [Option-2] PUSH & POP TEST                    PASS     |
+=========================================================+
|  TRANSACTION                                            |
+---------------------------------------------------------+
|    total                      :  1746                   |
|    reset                      :     0                   |
|    push (accepted)            :  1746                   |
|    pop  (accepted)            :  1745                   |
|    push ignored (full)        :     0                   |
|    pop  ignored (empty)       :     1                   |
+---------------------------------------------------------+
|  CHECK                                                  |
+---------------------------------------------------------+
|    pop_data vs expected       : PASS  1745   FAIL   0   |
|    full                       : PASS  1746   FAIL   0   |
|    empty                      : PASS  1746   FAIL   0   |
+---------------------------------------------------------+
|  DATA ACCEPT (0~255)                                    |
+---------------------------------------------------------+
|    end condition              : COVERED                 |
|    transactions               :  1746 / max 20000       |
|    push accepted (stored)     : 256 / 256               |
|    pop  checked  (PASS)       : 256 / 256               |
+---------------------------------------------------------+
|  COVERAGE                                               |
+---------------------------------------------------------+
|    push_data                  : 100.0 %                 |
|    pop_data                   : 100.0 %                 |
|    push / pop                 :  25.0 %                 |
|    empty / full               :  66.7 %                 |
|    fifo count (0~3)           :  40.0 %                 |
|    push/pop x empty/full      :  16.7 %                 |
|    Depth x Push / Pop         :  10.0 %                 |
+=========================================================+
```

**253 → 256**

pop 확인 (PASS) (0 ~ 255)

**3 → 0**

누락된 값

**1,000 → 1,746**

총 transaction (종료 시점)

첫 pop만 empty라 무시 (1회)
이후 매 cycle push + pop

# 적용 결과 Option-3 RANDOM without Reset: 213 → 256 / 256

04

**결론**

- 수정 전 run_count 1,000 : pop 확인 (PASS) 213 / 256 → 43개 누락
- 수정 후 3,130 cycle에서 256 / 256 확인 후 자동 종료 (COVERED), FAIL 0

## 수정 전 · tb_fifo.sv

```
+=============================================================+
|  [Option-3] RANDOM without Reset TEST          PASS         |
+=============================================================+
|  TRANSACTION                                                |
+-------------------------------------------------------------+
|    total                        :  1000                     |
|    reset                        :     0                     |
|    push (accepted)              :   445                     |
|    pop  (accepted)              :   443                     |
|    push ignored (full)          :    49                     |
|    pop  ignored (empty)         :    69                     |
+-------------------------------------------------------------+
|  CHECK                                                      |
+-------------------------------------------------------------+
|    pop_data vs expected         : PASS   443   FAIL   0     |
|    full                         : PASS  1000   FAIL   0     |
|    empty                        : PASS  1000   FAIL   0     |
+-------------------------------------------------------------+
|  MISSING DATA CHECK (0~255)                                 |
+-------------------------------------------------------------+
|    push accepted (stored)       : 213 / 256                 |
|    pop  checked  (PASS)         : 213 / 256                 |
+-------------------------------------------------------------+
|  COVERAGE                                                   |
+-------------------------------------------------------------+
|    push_data                    :  86.3 %                   |
|    pop_data                     :  83.2 %                   |
|    push / pop                   : 100.0 %                   |
|    empty / full                 : 100.0 %                   |
|    fifo count (0~3)             : 100.0 %                   |
|    push/pop x empty/full        : 100.0 %                   |
+=============================================================+
```

## 수정 후 · tb_fifo_pre.sv

```
+=============================================================+
|  [Option-3] RANDOM without Reset TEST          PASS         |
+=============================================================+
|  TRANSACTION                                                |
+-------------------------------------------------------------+
|    total                        :  3130                     |
|    reset                        :     0                     |
|    push (accepted)              :  1385                     |
|    pop  (accepted)              :  1383                     |
|    push ignored (full)          :   176                     |
|    pop  ignored (empty)         :   179                     |
+-------------------------------------------------------------+
|  CHECK                                                      |
+-------------------------------------------------------------+
|    pop_data vs expected         : PASS  1383   FAIL   0     |
|    full                         : PASS  3130   FAIL   0     |
|    empty                        : PASS  3130   FAIL   0     |
+-------------------------------------------------------------+
|  DATA ACCEPT (0~255)                                        |
+-------------------------------------------------------------+
|    end condition                : COVERED                   |
|    transactions                 :  3130 / max 20000         |
|    push accepted (stored)       : 256 / 256                 |
|    pop  checked  (PASS)         : 256 / 256                 |
+-------------------------------------------------------------+
|  COVERAGE                                                   |
+-------------------------------------------------------------+
|    push_data                    : 100.0 %                   |
|    pop_data                     : 100.0 %                   |
|    push / pop                   : 100.0 %                   |
|    empty / full                 : 100.0 %                   |
|    fifo count (0~3)             : 100.0 %                   |
|    push/pop x empty/full        : 100.0 %                   |
|    Depth x Push / Pop           : 100.0 %                   |
+=============================================================+
```

**213 → 256**

pop 확인 (PASS) (0 ~ 255)

**43 → 0**

누락된 값

**1,000 → 3,130**

총 transaction (종료 시점)

full 상태 push 무시 176회
empty 상태 pop 무시 179회

# 적용 결과 Option-4 RANDOM with Reset: 177 → 256 / 256

04

결론

- 수정 전 run_count 1,000 : pop 확인 (PASS) 177 / 256 → 79개 누락
- 수정 후 4,622 cycle에서 256 / 256 확인 후 자동 종료 (COVERED), FAIL 0

## 수정 전 · tb_fifo.sv

```
+=============================================================+
|  [Option-4] RANDOM with Reset TEST              PASS        |
+=============================================================+
|  TRANSACTION                                                |
+-------------------------------------------------------------+
|    total                        :  1000                     |
|    reset                        :   114                     |
|    push (accepted)              :   414                     |
|    pop  (accepted)              :   292                     |
|    push ignored (full)          :    12                     |
|    pop  ignored (empty)         :   153                     |
+-------------------------------------------------------------+
|  CHECK                                                      |
+-------------------------------------------------------------+
|    pop_data vs expected         : PASS   292   FAIL   0     |
|    full                         : PASS  1000   FAIL   0     |
|    empty                        : PASS  1000   FAIL   0     |
+-------------------------------------------------------------+
|  MISSING DATA CHECK (0~255)                                 |
+-------------------------------------------------------------+
|    push accepted (stored)       : 215 / 256                 |
|    pop  checked  (PASS)         : 177 / 256                 |
+-------------------------------------------------------------+
|  COVERAGE                                                   |
+-------------------------------------------------------------+
|    push_data                    :  84.8 %                   |
|    pop_data                     :  74.6 %                   |
|    push / pop                   : 100.0 %                   |
|    empty / full                 : 100.0 %                   |
|    fifo count (0~3)             : 100.0 %                   |
|    push/pop x empty/full        : 100.0 %                   |
+=============================================================+
```

## 수정 후 · tb_fifo_pre.sv

```
+=============================================================+
|  [Option-4] RANDOM with Reset TEST              PASS        |
+=============================================================+
|  TRANSACTION                                                |
+-------------------------------------------------------------+
|    total                        :  4622                     |
|    reset                        :   487                     |
|    push (accepted)              :  1978                     |
|    pop  (accepted)              :  1368                     |
|    push ignored (full)          :    77                     |
|    pop  ignored (empty)         :   675                     |
+-------------------------------------------------------------+
|  CHECK                                                      |
+-------------------------------------------------------------+
|    pop_data vs expected         : PASS  1368   FAIL   0     |
|    full                         : PASS  4622   FAIL   0     |
|    empty                        : PASS  4622   FAIL   0     |
+-------------------------------------------------------------+
|  DATA ACCEPT (0~255)                                        |
+-------------------------------------------------------------+
|    end condition                : COVERED                   |
|    transactions                 :  4622 / max 20000         |
|    push accepted (stored)       : 256 / 256                 |
|    pop  checked  (PASS)         : 256 / 256                 |
+-------------------------------------------------------------+
|  COVERAGE                                                   |
+-------------------------------------------------------------+
|    push_data                    : 100.0 %                   |
|    pop_data                     : 100.0 %                   |
|    push / pop                   : 100.0 %                   |
|    empty / full                 : 100.0 %                   |
|    fifo count (0~3)             : 100.0 %                   |
|    push/pop x empty/full        : 100.0 %                   |
|    Depth x Push / Pop           : 100.0 %                   |
+=============================================================+
```

**177 → 256**

pop 확인 (PASS) (0 ~ 255)

**79 → 0**

누락된 값

**1,000 → 4,622**

총 transaction (종료 시점)

동작 중 reset 487회
reset으로 버려진 값은 다시 뽑힐 때까지 진행

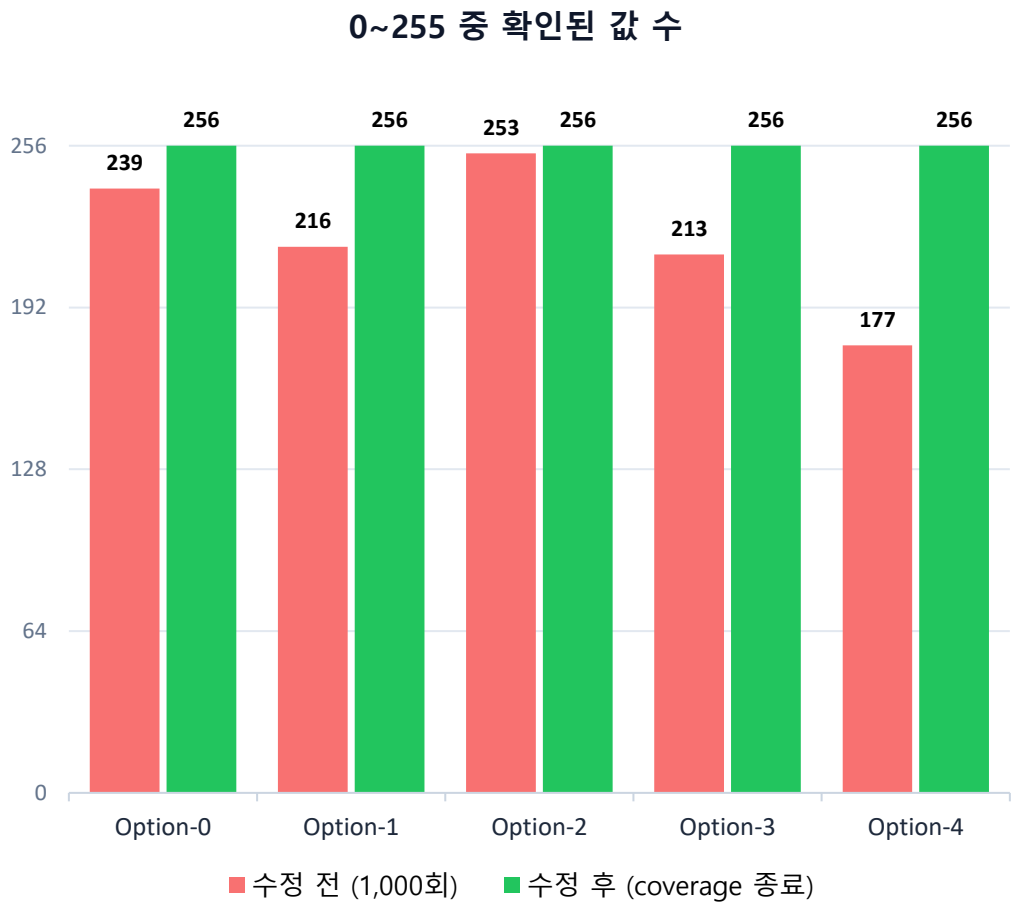
# 적용 결과 요약: 5개 Option 모두 0~255 full coverage 달성

04

결론

- 수정 전 run_count 1,000은 모든 Option에서 3~79개 값 누락, 수정 후 모두 256 / 256
- 필요한 횟수는 Option마다 1,746 ~ 4,622로 달라 고정 횟수로는 보장할 수 없음 → coverage 종료가 맞는 방식

| Option | 수정 전 확인 | 누락 | 수정 후 총 횟수 | 수정 후 확인 | 종료 |
|---|---|---|---|---|---|
| Option-0 PUSH ONLY | 239 / 256 | 17 | 2,101 | 256 / 256 | COVERED |
| Option-1 POP ONLY | 216 / 256 | 40 | 2,906 | 256 / 256 | COVERED |
| Option-2 PUSH & POP | 253 / 256 | 3 | 1,746 | 256 / 256 | COVERED |
| Option-3 RANDOM without Reset | 213 / 256 | 43 | 3,130 | 256 / 256 | COVERED |
| Option-4 RANDOM with Reset | 177 / 256 | 79 | 4,622 | 256 / 256 | COVERED |




· 확인 기준 : Option-0은 저장(push), Option-1~4는 pop 해서 PASS한 값
· 수정 전 횟수는 모두 1,000, 수정 후는 0~255를 다 확인한 시점

# 최종 결과: 5 / 5 Option PASS, FAIL 0, 0~255 full coverage

05

결론

- 모든 Option에서 pop_data · full · empty 비교 FAIL 0, assertion 위반 0, push · pop data coverage 100%
- Depth × push/pop cross는 Option-0~2가 정해진 조합만 만들어 낮고, 무작위 Option-3 · 4에서 100%

| 5 / 5 | 0 | 0 | 256 / 256 |
|---|---|---|---|
| Option PASS | FAIL (data · full · empty) | assertion 위반 | 0~255 확인 (모든 Option) |

| Option | 총 횟수 | reset | pop_data PASS / FAIL | full · empty FAIL | push / pop data cov | push/pop × empty/full | Depth × push/pop | 결과 |
|---|---|---|---|---|---|---|---|---|
| Option-0 PUSH ONLY | 2,101 | 350 | 0 / 0 | 0 | 100.0 / 0.0 | 33.3 % | 30.0 % | PASS |
| Option-1 POP ONLY | 2,906 | 0 | 1,292 / 0 | 0 | 100.0 / 100.0 | 41.7 % | 45.0 % | PASS |
| Option-2 PUSH & POP | 1,746 | 0 | 1,745 / 0 | 0 | 100.0 / 100.0 | 16.7 % | 10.0 % | PASS |
| Option-3 RANDOM without Reset | 3,130 | 0 | 1,383 / 0 | 0 | 100.0 / 100.0 | 100.0 % | 100.0 % | PASS |
| Option-4 RANDOM with Reset | 4,622 | 487 | 1,368 / 0 | 0 | 100.0 / 100.0 | 100.0 % | 100.0 % | PASS |

· Option-0은 pop이 없어 pop_data coverage 0% (의도), Option-2는 개수 1 유지라 Depth × push/pop 10% (의도)